

Simulador del planificador de CPU para SIGET

por: JOSE DAVID CELIS CEPEDA

1. objetivos y decisiones del diseño

El objetivo de este proyecto fue desarrollar un simulador del planificador de CPU para el Sistema Inteligente de Gestión del Tráfico (SIGET), con el propósito de representar cómo un sistema operativo administra diferentes procesos mediante algoritmos de planificación. La simulación busca mostrar el comportamiento de los procesos desde su creación hasta su finalización, permitiendo observar los estados por los que transitan y comparar dos estrategias de asignación del procesador.

para el diseño del programa se utilizó el lenguaje python por su facilidad para representar estructuras de datos y desarrollar simulaciones de forma clara. Se definieron cuatro procesos inspirados en tareas propias de SIGET: detección de accidentes, análisis de tráfico, actualización de semáforos y generación de estadísticas. Cada proceso cuenta con atributos como identificador, tiempo de irrupción, prioridad de alerta, tamaño de datos, tiempo de ejecución y tiempo restante.

tambien se implementó un sistema de estados que representa el ciclo de vida de cada proceso mediante los estados **nuevo**, **listo**, **En ejecución**, **bloqueado** y **terminado**. También se incluyó una simulación de bloqueo para representar la espera de recursos externos, como la recepción de datos provenientes de sensores de tráfico.

2. algoritmos utilizados y observaciones

Round Robin

este algoritmo utiliza un quantum de dos unidades de tiempo, asignando la CPU a cada proceso por un tiempo limitado antes de devolverlo a la cola de procesos listos si aún no ha terminado. Durante la simulación se observa cómo los procesos ingresan a la cola según su tiempo de irrupción, alternan el uso del procesador y permiten una distribución equilibrada del tiempo de CPU. Además, el proceso de detección de accidentes experimenta un estado de bloqueo temporal mientras espera información de sensores, regresando posteriormente al estado Listo para continuar su ejecución.

Algoritmo de prioridad

en este algoritmo, la CPU selecciona el proceso disponible con mayor prioridad, considerando que un número menor representa una prioridad más alta. La detección de accidentes, al tener prioridad 1, es atendida primero y completa toda su ejecución antes de permitir la entrada del siguiente proceso. Esta implementación corresponde a una planificación no apropiativa, ya que el proceso en ejecución no es interrumpido hasta finalizar.

despues de ejecutar el algoritmo, Los resultados muestran que Round Robin favorece la multitarea al repartir el tiempo de ejecución entre todos los procesos, mientras que el algoritmo de Prioridad ofrece una respuesta más rápida para tareas críticas, aunque puede retrasar procesos de menor importancia.

3. conclusiones

ambos algoritmos aportan ventajas según el escenario operativo, demostrando que la selección de una estrategia de planificación influye directamente en la eficiencia del procesamiento y en la capacidad de respuesta ante situaciones críticas.